KevaBook Software Architecture Document (SAD)

Table of Contents

1. Introduction

1.1 Purpose

This Software Architecture Document (SAD) describes the architectural design of the KevaBook platform. It defines the high-level structure of the system, its major software components, architectural principles, communication patterns, and technology choices that collectively satisfy the requirements specified in the Software Requirements Specification (SRS).

The purpose of this document is to provide software developers, architects, testers, and project stakeholders with a comprehensive understanding of how the KevaBook platform is structured and how its components interact to deliver the required functionality.

1.2 Scope

This document covers the software architecture of the KevaBook platform, including:

- Overall architectural style
- Microservice decomposition
- Service responsibilities
- Domain-Driven Design (DDD) boundaries
- Data architecture
- Inter-service communication
- Security architecture
- Deployment architecture
- Architectural decisions and design principles

Business requirements, domain entities, and business rules are documented separately in their respective documents.

1.3 Intended Audience

This document is intended for:

- Software Architects
- Software Developers
- System Integrators
- DevOps Engineers
- Quality Assurance Engineers
- Project Supervisors
- Technical Reviewers

1.4 Architectural Goals

The architecture of KevaBook has been designed to achieve the following objectives:

- Support a scalable multi-tenant Software-as-a-Service (SaaS) platform.
- Provide clear separation of business capabilities through a microservices architecture.
- Promote maintainability through loose coupling and high cohesion.
- Enable independent deployment and scalability of services.
- Ensure reliable communication between services using synchronous and asynchronous messaging.

- Support secure authentication and authorization.
- Maintain strict tenant isolation and data ownership.
- Facilitate future platform enhancements with minimal impact on existing services.

2. Architecture Overview

This section presents the overall architectural design of the KevaBook platform. It describes the architectural style, design principles, and high-level structure adopted to satisfy the functional and non-functional requirements defined in the Software Requirements Specification (SRS).

The architecture has been designed to support scalability, maintainability, security, and reliability while enabling independent evolution of the platform's business capabilities.

2.1 Architectural Style

KevaBook adopts a **Microservices Architecture** in which the platform is decomposed into a collection of small, autonomous services. Each microservice is responsible for a single business capability, owns its own data, and can be developed, deployed, and scaled independently.

The platform also incorporates an Event-Driven Architecture (EDA) for asynchronous communication between services. Business events are published whenever significant operations occur, allowing other services to react without creating tight coupling between components.

This combination of microservices and event-driven communication provides a flexible and extensible architecture suitable for a Software-as-a-Service (SaaS) booking platform.

2.2 Architectural Characteristics

The KevaBook architecture is designed around the following characteristics:

Scalability

Each microservice can be deployed and scaled independently based on workload demands. This allows services experiencing higher traffic, such as the Booking Service, to scale without affecting other services.

Maintainability

Business capabilities are separated into independent services with clearly defined responsibilities. This reduces coupling and simplifies maintenance and future enhancements.

Loose Coupling

Services communicate through well-defined interfaces and asynchronous events, minimizing dependencies between components.

High Cohesion

Each microservice encapsulates a single business capability, ensuring that related functionality is grouped together within the same service.

Fault Isolation

Failures within one microservice should not prevent unrelated services from continuing to operate whenever possible.

Extensibility

The architecture supports the addition of new services and business capabilities with minimal impact on existing components.

2.3 High-Level Architecture

The KevaBook platform consists of several independent microservices that collaborate to deliver the overall booking functionality.

The primary architectural components include:

- API Gateway
- Eureka Service Discovery
- Configuration Service
- User Service
- Business Service
- Service Catalog Service
- Booking Service
- Notification Service
- Message Broker
- Dedicated databases for each microservice

External clients communicate with the platform exclusively through the API Gateway. The gateway is responsible for routing requests to the appropriate microservices.

Business operations that require immediate responses are performed using synchronous REST communication, while background processes such as notifications are handled asynchronously through domain events.

2.4 Architectural Design Principles

The KevaBook architecture follows the following design principles:

Single Responsibility Principle

Each microservice is responsible for one business capability only.

Database per Service

Each microservice owns its own database and manages its own persistent data. Direct database access between services is not permitted.

Domain Ownership

Every business capability has a clearly defined owner responsible for maintaining its business logic and data.

API-First Communication

Synchronous interactions between services occur through well-defined REST APIs.

Event-Driven Integration

Business events are used to notify interested services of significant domain changes without introducing tight coupling.

Independent Deployment

Each microservice can be built, deployed, and updated independently of the others.

Stateless Services

Application services are designed to remain stateless, enabling horizontal scaling and simplified deployment.

Multi-Tenant Isolation

All business operations are executed within the context of a single Business tenant, ensuring logical separation of data between independent businesses.

2.5 Architectural Benefits

The selected architecture provides several benefits for the KevaBook platform, including:

- Independent development of business capabilities.
- Improved scalability through service-level scaling.
- Simplified maintenance due to service isolation.
- Increased resilience through fault isolation.
- Flexible integration using asynchronous messaging.
- Easier introduction of future platform features.
- Clear separation of business domains.
- Support for continuous deployment and incremental system evolution.

3. Architectural Views

This section presents the KevaBook architecture from multiple viewpoints. Each architectural view focuses on a specific aspect of the system, allowing stakeholders to understand how the platform is organized, how components interact, how data is managed, and how the system is deployed. Together, these views provide a comprehensive understanding of the overall software architecture.

3.1 Logical View

The Logical View describes the major software components of the KevaBook platform and their responsibilities. It focuses on the functional decomposition of the system rather than its physical deployment.

The platform is organized around independently deployable microservices, each responsible for a distinct business capability.

The primary logical components are:

- API Gateway
- Configuration Service
- Service Discovery
- User Service
- Business Service
- Service Catalog Service
- Booking Service
- Notification Service
- Message Broker
- Dedicated databases for each microservice

Each microservice owns its business logic and persistent data and communicates with other services through well-defined interfaces.

The detailed responsibilities of each component are described in the **System Components** section of this document.

3.2 Process View

The Process View describes how the various components collaborate to fulfil business operations.

KevaBook employs two communication models:

Synchronous Communication

Synchronous communication is used when an immediate response is required. Services communicate using REST APIs through the API Gateway. However for interservice communication, services call each other directly using the HTTP REST APIs(Json over Https).

Typical synchronous operations include:

- Business management
- Staff management
- Service management
- Booking creation and retrieval

Asynchronous Communication

Asynchronous communication is used for background processing and event propagation. When significant business events occur, the originating service publishes an event that can be consumed by interested services.

Typical asynchronous operations include:

- Booking confirmation notifications
- Booking cancellation notifications
- Reminder notifications
- Future analytics and reporting events

This communication model reduces coupling between services while improving scalability and resilience.

3.3 Data View

The Data View describes how data is organized and owned across the platform.

KevaBook follows the **Database per Service** architectural pattern, where each microservice exclusively owns and manages its own persistent data.

Key principles include:

- Every microservice has its own dedicated database.
- Services do not directly access another service's database.
- Data sharing occurs only through service APIs or published domain events.
- Each service is responsible for maintaining the integrity of its own data.

This approach promotes loose coupling, independent evolution, and service autonomy.

3.4 Deployment View

The Deployment View illustrates how the KevaBook platform is deployed within its runtime environment.

The platform is designed as a distributed cloud-native application composed of independently deployable services.

Major deployment components include:

- API Gateway
- Configuration Service
- Service Discovery
- Individual microservices
- Message Broker
- Dedicated PostgreSQL databases
- External Email Provider

- External SMS Provider

Each service can be deployed, updated, monitored, and scaled independently, allowing the platform to accommodate increasing workloads while minimizing operational impact on other services.

The detailed deployment topology is presented later in the **Deployment Architecture** section.

3.5 Technology Mapping

component	Technology	Purpose
API Gateway	Springcloud gateway	Single entry point for client requests
Service Discovery	Netflix Eureka	Dynamic service registration and discovery
Configuration Service	Spring Cloud Config	Centralized configuration management
User Service	Springboot	Business owner management
Business Service	Springboot	Business, staff, and location management
Service Catalog Service	Springboot	Service management
Booking service	Springboot	Booking and scheduling
Notification Service	Springboot	Email and SMS notifications
Message Broker	RabbitMq	Asynchronous event-driven communication
Database	Postgres	Persistent storage (database per service)
Authentication	Keycloak	Identity and access management
Containerization	Docker	Application packaging and deployment

4. System Components

This section describes the major software components that constitute the KevaBook platform. Each component is responsible for a specific architectural or business capability and collaborates with other components through well-defined interfaces. The platform follows the microservices architectural style, where each service is independently deployable, owns its business logic, and manages its own persistent data.

4.1 API Gateway

The API Gateway serves as the single entry point to the KevaBook platform. All external client requests pass through the gateway before reaching the appropriate microservice.

Responsibilities

- Receive incoming HTTP requests from clients.
- Route requests to the appropriate microservice.
- Validate authentication tokens before forwarding requests.
- Enforce authorization policies.
- Apply cross-cutting concerns such as CORS, rate limiting, and request filtering.
- Return responses from downstream services to clients.

Communication

- Receives HTTPS requests from web and mobile clients.
- Routes requests to backend microservices using HTTP REST interfaces.

4.2 Configuration Service

The Configuration Service provides centralized configuration management for all microservices.

Responsibilities

- Store shared application configuration.
- Provide configuration during service startup.
- Maintain environment-specific configuration.
- Reduce duplication of configuration across services.

Communication

- Supplies configuration to registered microservices during startup.

4.3 Service Discovery

The Service Discovery component enables dynamic service registration and discovery.

Responsibilities

- Register active microservice instances.
- Maintain a registry of available services.
- Allow services to discover one another dynamically.
- Support load balancing across multiple service instances.

Communication

- Microservices register themselves during startup.
- Services query the registry to locate other services.

4.4 User Service

The User Service manages business owner accounts and access to the platform.

Responsibilities

- Manage Business Owner accounts.
- Maintain user profile information.
- Associate users with Businesses.
- Support user-related business operations.

Owned Data

- Business Owners
- User Profiles
- Business Associations

Communication

- Exposes HTTP REST APIs.
- Owns its dedicated PostgreSQL database.

4.5 Business Service

The Business Service manages tenant-specific business information.

Responsibilities

- Manage Businesses.
- Manage Staff members.
- Manage Locations.
- Manage Business Settings.
- Provide staff information to other services.

Owned Data

- Businesses
- Staff
- Locations
- Business Settings

Communication

- Exposes HTTP REST APIs.
- Provides business and staff information to authorized services.

- Owns its dedicated PostgreSQL database.

4.6 Service Catalog Service

The Service Catalog Service manages the services offered by each Business.

Responsibilities

- Create and manage Services.
- Maintain service pricing.
- Maintain service duration.
- Manage service availability status.

Owned Data

- Services
- Service Pricing
- Service Duration
- Service Status

Communication

- Exposes HTTP REST APIs.
- Provides service information to the Booking Service.
- Owns its dedicated PostgreSQL database.

4.7 Booking Service

The Booking Service is the core transactional component of the KevaBook platform. It manages appointment reservations and enforces the primary business rules governing the booking lifecycle.

Responsibilities

- Create bookings.
- Update bookings.
- Cancel bookings.
- Validate staff availability.
- Prevent overlapping bookings.

- Manage booking status transitions.
- Publish booking-related domain events.

Owned Data

- Bookings
- Booking Status
- Booking History

Communication

- Exposes HTTP REST APIs.
- Retrieves business and staff information through HTTP REST interfaces.
- Retrieves service information through HTTP REST interfaces.
- Publishes domain events to the message broker after successful booking operations.
- Owns its dedicated PostgreSQL database.

4.8 Notification Service

The Notification Service is responsible for delivering notifications to clients and business owners.

Responsibilities

- Process booking-related events.
- Send email notifications.
- Send SMS notifications.
- Record notification delivery status.
- Retry failed notification deliveries where appropriate.

Owned Data

- Notification Records
- Delivery Status
- Notification History

Communication

- Consumes domain events from the message broker.
- Communicates with external email and SMS providers.
- Owns its dedicated PostgreSQL database.

4.9 Message Broker

The Message Broker provides asynchronous communication between microservices using an event-driven messaging model.

Responsibilities

- Receive published domain events.
- Route events to subscribed services.
- Decouple event producers from event consumers.
- Improve scalability and fault isolation.

Published Events (Initial Version)

- BookingCreated
- BookingCancelled
- BookingCompleted

Event Consumers

- Notification Service

Future platform versions may introduce additional event consumers such as analytics, reporting, auditing, and payment services.

5. Domain-Driven Design (DDD)

This section describes how Domain-Driven Design (DDD) principles are applied within the KevaBook architecture. The platform is organized into bounded contexts that align with distinct business capabilities. Each bounded context is implemented as an independent microservice responsible for its own business logic, data, and consistency rules.

The detailed business entities, attributes, relationships, and domain rules are documented separately in the **KevaBook Domain Model** and **KevaBook Business Rules** documents. This section focuses on the architectural realization of those concepts.

5.1 Bounded Contexts

The KevaBook platform is divided into the following bounded contexts.

User Management Context

Implemented by: User Service

Responsibilities

- Manage Business Owner accounts.
- Maintain user profile information.
- Associate users with Businesses.
- Support authentication integration.

Owned Domain Objects

- Business Owner
- User Profile

Business Management Context

Implemented by: Business Service

Responsibilities

- Manage Businesses.
- Manage Staff members.

- Manage Locations.
- Manage Business Settings.

Owned Domain Objects

- Business
- Staff
- Location
- Business Settings

Service Catalog Context

Implemented by: Service Catalog Service

Responsibilities

- Manage bookable Services.
- Maintain service pricing.
- Maintain service duration.
- Maintain service availability status.

Owned Domain Objects

- Service

Booking Context

Implemented by: Booking Service

Responsibilities

- Create bookings.
- Validate booking requests.
- Prevent overlapping bookings.
- Manage booking lifecycle.
- Publish booking domain events.

Owned Domain Objects

- Booking

Notification Context

Implemented by: Notification Service

Responsibilities

- Process booking events.
- Send email notifications.
- Send SMS notifications.
- Maintain notification history.

Owned Domain Objects

- Notification
- Notification Delivery Status

5.2 Aggregate Ownership

Each bounded context owns one or more aggregates and is solely responsible for enforcing the business rules associated with those aggregates.

Aggregate	Owning service
Business	Business service
Booking	Booking service
Availability	Business service
Service Aggregate	Service Catalog Service
User Aggregate	User Service
Notification Aggregate	Notification Service

Each aggregate is the authoritative source for its data and business invariants. Other services may reference aggregate identifiers but must not modify another service's aggregate directly.

5.3 Domain Ownership Principles

The KevaBook architecture follows the principle of single ownership for domain data.

The following principles govern domain ownership:

- Each domain object belongs to exactly one bounded context.
- Each bounded context owns its business logic and persistent data.
- Direct database access between services is prohibited.
- Cross-service interactions occur only through well-defined HTTP REST interfaces or asynchronous domain events.
- Each service is responsible for maintaining the consistency of its own aggregates.

This approach ensures clear ownership, loose coupling, and independent evolution of the platform's business capabilities.

5.4 Cross-Context Communication

Although each bounded context is autonomous, collaboration between contexts is required to support complete business workflows.

KevaBook supports two communication patterns.

Synchronous Communication

Business data required during request processing is obtained through HTTP REST interfaces.

Typical examples include:

- Booking Service retrieving Staff information from the Business Service.
- Booking Service retrieving Service information from the Service Catalog Service.
- API Gateway forwarding client requests to the appropriate microservice.

Asynchronous Communication

Domain events are published when significant business operations occur.

Examples include:

- BookingCreated
- BookingCancelled
- BookingCompleted

These events are published by the Booking Service and consumed by the Notification Service to trigger email and SMS notifications.

This event-driven approach reduces coupling while improving scalability and fault isolation.

5.5 Ubiquitous Language

KevaBook adopts a shared business vocabulary across all bounded contexts to ensure consistent communication between developers, architects, testers, and business stakeholders.

Business Term	Definition
Business	A tenant registered on the KevaBook platform that provides one or more bookable services.
Business Owner	An authenticated user responsible for managing a Business within the platform.
Staff	An individual associated with a Business who delivers one or more Services.
Service	A bookable offering provided by a Business
Availability	The schedule defining when a Staff member is available to accept bookings.
Booking	A reservation made by a Client for a specific Service with an assigned Staff member.
client	An external customer who creates a booking through the platform.
Notification	An email or SMS message generated in response to a business event, such as booking creation or cancellation.

Maintaining a consistent ubiquitous language across all services promotes shared understanding and reduces ambiguity throughout the software development lifecycle.

6. Data Architecture

This section describes how data is organized, owned, stored, and managed within the KevaBook platform. The architecture follows the Database per Service pattern, where each microservice owns and manages its own persistent data. This approach promotes loose coupling, service autonomy, and independent scalability while ensuring clear ownership of business data.

The detailed database schemas, table definitions, entity relationships, indexes, and constraints are documented separately in the KevaBook Database Design document.

6.1 Data Architecture Overview

KevaBook adopts a decentralized data architecture in which every microservice maintains its own dedicated database. Each service is solely responsible for creating, updating, and maintaining the data within its database. Services collaborate through HTTP REST interfaces and asynchronous domain events rather than sharing database tables or directly querying another service's data.

This architectural approach enables independent deployment, simplified maintenance, and improved scalability.

6.2 Database per Service Pattern

Each microservice owns an independent PostgreSQL database. The platform currently consists of the following service databases:

Microservice	Database	Primary Data
User Service	User Database	Business Owners, User Profiles
Business Service	Business Database	Businesses, Staff, Locations, Business Settings
Service Catalog Service	Service Catalog Database	Services, Pricing, Duration
Booking Service	Booking Database	Bookings, Booking Status, Booking History
Notification Service	Notification Database	Notifications, Delivery Status, Notification History

The following principles apply:

- Each database is owned by exactly one microservice.
- No service may directly access another service's database.
- Data exchange occurs only through HTTP REST interfaces or published domain events.
- Database schemas evolve independently according to the needs of their owning service.

6.3 Data Ownership

Data ownership is fundamental to the KevaBook architecture.

Each business entity has a single authoritative owner responsible for maintaining its lifecycle and enforcing its business rules.

Domain Object	Owning Service
Business Owner	User Service
Business	Business Service
Staff	Business Service
Location	Business Service
Business Settings	Business Service
Service	Service Catalog Service
Booking	Booking Service
Notification	Notification Service

This ownership model prevents conflicting updates and establishes clear responsibility for each domain object.

6.4 Persistence Strategy

All microservices persist their operational data using PostgreSQL.

The persistence strategy follows these principles:

- Transactional operations are executed within the boundaries of a single service.
- Each service manages its own database schema.

- Database transactions do not span multiple services.
- Referential integrity is maintained within each service database.
- Cross-service relationships are represented using identifiers rather than database foreign keys.

This strategy aligns with microservices best practices by preserving service autonomy and avoiding distributed database transactions.

6.5 Data Consistency

KevaBook adopts an **eventual consistency** model for operations that span multiple services.

Business operations that occur entirely within a single service maintain strong transactional consistency.

When multiple services participate in a business process:

- The initiating service completes its local transaction.
- Relevant domain events are published.
- Interested services consume the events and update their own data where necessary.

This approach eliminates the need for distributed transactions while maintaining overall system consistency.

6.6 Multi-Tenant Data Isolation

KevaBook is a multi-tenant Software-as-a-Service (SaaS) platform.

Tenant isolation is achieved through logical separation of business data.

The following principles apply:

- Every business entity is associated with a Business identifier.
- Business data is accessible only within its owning tenant.
- Cross-tenant data access is prohibited.
- All service operations enforce tenant-aware data access controls.

This ensures that one Business cannot access or modify the data belonging to another Business.

6.7 Data Integrity

The platform maintains data integrity through a combination of application-level validation and database constraints.

Examples include:

- Prevention of overlapping bookings for the same Staff member.
- Validation of booking time ranges.
- Enforcement of mandatory business identifiers.
- Referential integrity within individual service databases.
- Validation of unique business owner email addresses.

Critical business rules are validated before data is persisted to ensure consistency and correctness.

6.8 Data Backup and Recovery

To ensure business continuity, each service database should be backed up independently.

The backup strategy should support:

- Periodic full database backups.
- Point-in-time recovery where supported.
- Secure storage of backup files.
- Restoration of individual service databases without affecting unrelated services.

Independent backups align with the database-per-service architecture and simplify disaster recovery procedures.

7. Inter-Service Communication

This section describes the communication mechanisms used between the microservices that comprise the KevaBook platform. The architecture employs both synchronous and asynchronous communication patterns to balance responsiveness, reliability, and loose coupling.

Synchronous communication is implemented using HTTP REST interfaces, while asynchronous communication is achieved through event-driven messaging using RabbitMQ.

7.1 Communication Overview

Microservices within the KevaBook platform communicate using two complementary communication models:

- **Synchronous Communication** – Used when an immediate response is required to complete a client request.
- **Asynchronous Communication** – Used for background processing and notification of business events.

The communication model selected depends on the business requirements of the operation being performed.

7.2 Synchronous Communication

KevaBook uses HTTP REST interfaces for synchronous communication between microservices. This communication model is appropriate for operations where a service requires an immediate response before continuing processing.

Typical examples include:

- Retrieving Business information.
- Retrieving Staff information.
- Retrieving Service details.
- Validating data required during booking creation.

Communication Principles

The following principles govern synchronous communication:

- Services communicate through well-defined HTTP REST APIs.
- Services never communicate through direct database access.
- Each request targets the owning service for the required business data.
- Services remain independent and expose only their public interfaces.
- Service discovery is used to locate service instances dynamically.

7.3 Asynchronous Communication

KevaBook adopts an event-driven architecture for operations that do not require an immediate response. When significant business events occur, the originating service publishes a domain event to RabbitMQ. Interested services subscribe to the relevant events and perform their own processing independently. This communication model reduces coupling between services while improving scalability and fault tolerance.

Initial Domain Events

The Booking Service publishes the following events:

- BookingCreated
- BookingCancelled
- BookingCompleted

The Notification Service subscribes to these events to generate and deliver email and SMS notifications.

Future platform versions may introduce additional event consumers, including analytics, reporting, auditing, and payment services.

7.4 Communication Flow

A typical booking workflow involves both communication models.

1. A client submits a booking request through the API Gateway.
2. The Booking Service retrieves the required Business and Staff information using HTTP REST interfaces.
3. The Booking Service retrieves Service information from the Service Catalog Service using HTTP REST interfaces.
4. The Booking Service validates the booking request and persists the booking.
5. After a successful transaction, the Booking Service publishes a BookingCreated event to RabbitMQ.
6. The Notification Service consumes the event and sends the appropriate email and SMS notifications.

This hybrid communication model ensures that client-facing operations remain responsive while background tasks execute independently.

7.5 Service Discovery

KevaBook uses a service discovery mechanism to enable dynamic communication between microservices. Instead of using fixed network addresses, services register themselves with the Service Discovery component during startup. When one service needs to communicate with another, it queries the service registry to obtain the current service instance.

This approach provides several benefits:

- Dynamic service registration.
- Automatic service discovery.
- Improved fault tolerance.
- Support for load balancing.
- Simplified service deployment.

7.6 Communication Reliability

To improve reliability, the following communication principles are applied:

HTTP REST Communication

- Request validation is performed before processing.
- Appropriate HTTP status codes are returned for success and failure conditions.
- Service timeouts should be configured to prevent indefinite waiting.
- Temporary failures may be retried where appropriate.

Event Communication

- Domain events are published only after successful completion of the originating transaction.
- Event consumers process events independently.
- Failed event processing should support retry mechanisms.
- Event processing should be designed to be idempotent to prevent duplicate side effects.

These practices improve system resilience while maintaining consistency across independently deployed services.

7.7 Communication Principles

The KevaBook platform follows the following communication principles:

- Microservices communicate through well-defined interfaces.
- Direct database access between services is prohibited.
- HTTP REST interfaces are used for synchronous request-response interactions.
- RabbitMQ is used for asynchronous event-driven communication.
- Services remain loosely coupled and independently deployable.
- Business events are published after successful transactional operations.
- Each service owns and manages its own communication contracts.

7.8 Service Communication Matrix

The following matrix summarizes the communication relationships between the components of the KevaBook platform. It identifies the source component, target component, communication type, protocol, and the purpose of each interaction.

Source Component	Target component	Communication Type	Purpose	Protocol
Client Application	API Gateway	Synchronous	Submit requests to the platform	Http
API Gateway	User service	Synchronous	Manage Business Owner accounts and user information.	Http Rest
API Gateway	Business service	Synchronous	Manage Businesses, Staff, Locations, and Business Settings.	Http Rest
API Gateway	Service Catalog Service	Synchronous	Manage Services offered by Businesses.	Http Rest
API Gateway	Booking Service	Synchronous	Create, retrieve, update, and cancel Bookings	Http Rest
Booking Service	Business Service	Synchronous	Retrieve Business and Staff information required for booking validation.	Http Rest
Booking Service	Service Catalog Service	Synchronous	Retrieve Service details such as duration, pricing, and availability status.	Http Rest

Booking Service	RabbitMQ	Asynchronous	Publish booking domain events after successful transactions.	AMQP
RabbitMQ	Notification Service	Asynchronous	Deliver booking-related events for notification processing.	AMQP
Notification Service	Email Provider	Synchronous	Send booking confirmation, cancellation, and reminder emails	SMTP or Provider API
Notification Service	SMS Provider	Synchronous	Send booking confirmation, cancellation, and reminder SMS messages.	Https Api
All Microservices	Configuration Service	Synchronous	Retrieve centralized application configuration during startup.	Http Rest
All Microservices	Service Discovery	Synchronous	Register service instances and discover other services dynamically.	Http Rest

Communication Principles

The communication matrix is governed by the following architectural principles:

- All external client requests enter the platform through the API Gateway.
- Microservices communicate synchronously using HTTP REST interfaces when an immediate response is required.
- RabbitMQ is used exclusively for asynchronous, event-driven communication.
- Services never communicate through direct database access.
- Each microservice communicates only through published APIs or domain events.
- Service discovery enables dynamic location of service instances without hard-coded network addresses.
- Domain events are published only after successful completion of local transactions.
- Each service remains independently deployable and maintains ownership of its communication contracts.

8. Security Architecture

This section describes the security architecture of the KevaBook platform. The architecture is designed to protect business data, enforce tenant isolation, and secure access to platform resources while maintaining a scalable and maintainable microservices environment.

KevaBook adopts a centralized security model in which authentication and authorization are enforced at the API Gateway. Identity management is delegated to Keycloak, while the individual microservices focus exclusively on business logic.

8.1 Security Overview

The KevaBook platform follows the principle of centralized authentication and decentralized business processing. All external client requests enter the platform through the API Gateway. Before a request is forwarded to any backend microservice, the gateway validates the client's identity and determines whether the requested operation is authorized. Once a request has been successfully authenticated and authorized, it is forwarded to the appropriate microservice for business processing.

This architecture provides:

- Centralized security enforcement.
- Consistent authentication across all services.
- Reduced duplication of security logic.
- Simplified service implementation.

8.2 Authentication

Authentication is performed using Keycloak. Business Owners authenticate through Keycloak using their registered credentials. Upon successful authentication, Keycloak issues a JSON Web Token (JWT), which is included in subsequent requests to the API Gateway. The API Gateway validates the JWT before forwarding the request to the target microservice.

Guest users are not required to authenticate when accessing publicly available platform functionality, such as browsing Businesses or creating bookings where guest booking is supported.

8.3 Authorization

Authorization determines whether an authenticated user is permitted to perform a requested operation.

KevaBook implements Role-Based Access Control (RBAC) to restrict access to protected resources.

Typical roles include:

- Business Owner
- Administrator (future extension)

Authorization decisions are enforced by the API Gateway before requests are forwarded to backend services.

Protected operations include:

- Managing Businesses.
- Managing Staff.
- Managing Services.
- Managing Availability.
- Managing Bookings.
- Updating Business Settings.

Public operations include:

- Viewing available Businesses.
- Viewing Services.
- Creating guest bookings (where enabled).

8.4 API Gateway Security

The API Gateway acts as the platform's security enforcement point.

Its security responsibilities include:

- Validating JWT access tokens.
- Authenticating incoming requests.
- Authorizing access to protected resources.
- Routing requests to backend microservices.
- Rejecting unauthorized or invalid requests.
- Applying cross-cutting security policies.

By centralizing these responsibilities, backend microservices remain focused on implementing business functionality.

8.5 Service-to-Service Security

Communication between internal microservices occurs over HTTP REST interfaces within the trusted platform environment. Microservices rely on the API Gateway to authenticate external requests before they are processed. Services communicate only through published APIs and never through direct database access.

8.6 Multi-Tenant Security

KevaBook is a multi-tenant Software-as-a-Service (SaaS) platform. To protect tenant data, the platform enforces strict tenant isolation.

The following security principles apply:

- Every business resource belongs to exactly one Business tenant.
- Business Owners may access only the resources associated with their own Business.
- Cross-tenant data access is prohibited.
- All protected operations enforce tenant-aware authorization.
- Data ownership is preserved across all microservices.

These controls ensure confidentiality and integrity of tenant data throughout the platform.

9. Technology Stack

This section describes the technologies selected for implementing the KevaBook platform. The chosen technologies align with the project's architectural goals of scalability, maintainability, security, and independent service deployment. The platform follows a cloud-native microservices architecture implemented using Java and the Spring ecosystem.

9.1 Programming Language

Java

9.2 Application Framework

Springboot

9.3 Microservices Infrastructure

Technology	Purpose	Justification
Spring Cloud Gateway	API Gateway	Serves as the single entry point for client requests, providing request routing, authentication, authorization, and other cross-cutting concerns.
Netflix Eureka	Service Discovery	Enables automatic registration and discovery of microservice instances, eliminating hard-coded service locations.
Spring Cloud Config	Centralized Configuration	Provides centralized management of application configuration across all microservices and environments.

9.4 Security

Technology	Purpose	Justification
Keycloak	Identity and Access Management	Provides centralized authentication, authorization, user identity management, and JWT token issuance.
JWT (JSON Web Token)	Authentication Token	Enables secure, stateless authentication between clients and the platform.

9.5 Communication

Technology	Purpose	Justification
HTTP REST	Synchronous Communication	Supports request-response communication between clients and microservices, as well as inter-service communication requiring immediate responses.
RabbitMQ	Asynchronous Messaging	Enables event-driven communication between microservices, reducing coupling and improving scalability and reliability.
AMQP	Messaging Protocol	Standard protocol used by RabbitMQ for reliable message delivery.

9.6 Database Technologies

Technology	Purpose	Justification
PostgreSQL	Relational Database Management System	Stores transactional data with strong ACID guarantees, making it well suited for booking, scheduling, and business management. Each microservice owns its own PostgreSQL database following the Database-per-Service architectural pattern.
Spring Data JPA	Data Access Layer	Simplifies repository implementation while providing consistent access to relational databases.
Hibernate	ORM Framework	Maps Java entities to relational database tables and manages persistence efficiently.

9.7 Build and Dependency Management

Technology	Purpose	Justification
Apache Maven	Build Automation	Manages project dependencies, compilation, testing, packaging, and deployment of all microservices.

9.8 Containerization and Deployment

Docker

Kubernetes (future)

10. Deployment Architecture

This section describes how the KevaBook platform is deployed and executed within its runtime environment. The deployment architecture defines the physical arrangement of the system components, supporting infrastructure, communication pathways, and deployment strategy.

KevaBook is designed as a cloud-native microservices application where each service can be deployed, updated, and scaled independently. The architecture promotes high availability, maintainability, and operational flexibility while supporting future horizontal scalability.

10.1 Deployment Overview

The KevaBook platform follows a distributed deployment model based on independently deployable microservices. Each microservice executes as an isolated application instance and owns its own database. Supporting infrastructure services, including the API Gateway, Configuration Service, Service Discovery, RabbitMQ, and PostgreSQL databases, provide the runtime environment required for the platform.

External clients interact with the platform exclusively through the API Gateway, which routes requests to the appropriate backend services.

10.2 Deployment Environment

The platform is designed to support multiple deployment environments throughout the software development lifecycle.

Environment	Purpose
Development	Local software development and debugging.
Testing	Functional, integration, and system testing.
Staging	Pre-production validation in an environment closely matching production.
Production	Live deployment serving Business Owners and Clients.

Containerization using Docker ensures consistent execution across all deployment environments.

10.3 Deployment Components

The runtime environment consists of the following components:

Infrastructure Components

- API Gateway
- Configuration Service
- Service Discovery
- RabbitMQ Message Broker
- PostgreSQL Databases

Business Microservices

- User Service
- Business Service
- Service Catalog Service
- Booking Service
- Notification Service

External Services

- Keycloak Identity Provider
- Email Service Provider
- SMS Service Provider

Each component performs a dedicated responsibility and communicates through well-defined interfaces.

10.4 Deployment Topology

The KevaBook platform is deployed as a collection of independent Docker containers connected through an internal network.

The deployment topology consists of:

- Client applications accessing the platform through HTTPS.
- API Gateway serving as the single entry point.
- Service Discovery enables dynamic service registration.
- Configuration Service supplying centralized configuration.
- Independent business microservices.
- RabbitMQ facilitating asynchronous messaging.

- Dedicated PostgreSQL databases for each microservice.
- External identity, email, and SMS providers.

A deployment diagram illustrating the runtime topology is provided within this section.

10.5 Scalability Strategy

The deployment architecture supports independent horizontal scaling of individual services.

Key scalability characteristics include:

- Each microservice can be replicated independently based on workload.
- Service Discovery automatically registers multiple service instances.
- API Gateway distributes incoming requests across available service instances.
- RabbitMQ supports asynchronous workload distribution between event producers and consumers.
- Database resources can be scaled independently according to service requirements.

This architecture allows heavily utilized services, such as the Booking Service, to scale without affecting unrelated services.

10.6 High Availability and Fault Tolerance

KevaBook is designed to maximize service availability while minimizing the impact of component failures.

The architecture supports:

- Independent deployment of each microservice.
- Isolation of service failures.
- Dynamic service registration and discovery.
- Retry mechanisms for asynchronous message processing.
- Independent database ownership.
- Container restart and recovery mechanisms provided by the deployment platform.

Future production deployments may introduce multiple instances of critical services to eliminate single points of failure.

10.7 Deployment Considerations

Successful deployment of the KevaBook platform requires appropriate operational practices.

Deployment considerations include:

- Environment-specific configuration managed through the Configuration Service.
- Secure management of application secrets and credentials.
- Database schema versioning using Flyway.
- HTTPS for secure client communication.
- Centralized logging and monitoring.
- Regular database backups.
- Health checks for all deployed services.
- Controlled deployment and rollback procedures.

These considerations ensure that the platform remains secure, maintainable, and reliable throughout its operational lifecycle.

11. Architectural Decisions (ADR)

This section documents the key architectural decisions made during the design of the KevaBook platform. Each decision records the selected approach and the rationale behind it, providing context for future development, maintenance, and architectural evolution.

11.1 Architectural Decision Summary

Decision ID	Architectural Decision	Rationale
ADR-001	Adopt a Microservices Architecture	Enables independent development, deployment, scalability, and maintenance of business capabilities.
ADR-002	Use Domain-Driven Design (DDD)	Aligns software boundaries with business domains, improving maintainability and reducing coupling.
ADR-003	Implement Database-per-Service	Ensures service autonomy, clear data ownership, and independent schema evolution.

ADR-004	Use PostgreSQL as the primary database	Provides ACID-compliant transactions, strong consistency, and reliable support for booking and scheduling data.
ADR-005	Use HTTP REST for synchronous communication	Enables efficient request-response communication between microservices when immediate responses are required.
ADR-006	Use RabbitMQ for asynchronous communication	Supports event-driven processing, loose coupling, and scalable background operations such as notifications.
ADR-007	Centralize security using Keycloak	Provides standardized authentication, authorization, identity management, and JWT token issuance.
ADR-008	Enforce authentication and authorization at the API Gateway	Centralizes security enforcement and allows backend services to focus on business logic.
ADR-009	Use Spring Cloud Gateway	Provides centralized request routing, security enforcement, and cross-cutting concerns for client requests.
ADR-010	Use Netflix Eureka for Service Discovery	Enables dynamic service registration and discovery without hard-coded network addresses.
ADR-011	Use Spring Cloud Config	Centralizes application configuration management across all microservices and deployment environments.
ADR-012	Containerize services using Docker	Ensures consistent deployment, portability, and environment independence.
ADR-013	Use Flyway for database schema migrations	Provides version-controlled, repeatable, and reliable database schema management.
ADR-014	Support guest booking	Reduces barriers for clients by allowing bookings without requiring user registration.
ADR-015	Support multi-staff scheduling	Enables Businesses to assign bookings to multiple Staff members, improving flexibility and scalability for service providers.

11.2 Architectural Principles

The architectural decisions presented in this document are guided by the following principles:

- Single Responsibility Principle (SRP)
- Separation of Concerns
- Loose Coupling
- Service Autonomy
- Domain Ownership
- Database-per-Service
- Event-Driven Communication
- Scalability
- Maintainability
- Security by Design
- Multi-Tenant Isolation

These principles ensure that the KevaBook platform remains modular, extensible, secure, and capable of evolving as new business requirements emerge.

11.3 Trade-Offs and Design Considerations

The selected architecture introduces several trade-offs that were considered during the design process.

Microservices Complexity

Adopting a microservices architecture increases operational complexity due to service discovery, distributed communication, monitoring, and deployment. However, these challenges are outweighed by the benefits of scalability, independent deployment, and clear separation of business capabilities.

Eventual Consistency

Using the Database-per-Service pattern eliminates distributed database transactions. As a result, operations spanning multiple services rely on eventual consistency through asynchronous messaging. This trade-off improves scalability and service autonomy while requiring careful handling of distributed workflows.

Centralized Security

Enforcing authentication and authorization at the API Gateway simplifies backend services and provides consistent security policies. Microservices remain responsible for validating business-specific rules, including tenant ownership and domain constraints.

Independent Databases

Maintaining separate databases for each microservice increases infrastructure requirements but significantly improves service autonomy, fault isolation, and independent schema evolution.

11.4 Architectural Evolution

The KevaBook architecture is intentionally designed to support future growth.

Future architectural enhancements may include:

- Additional microservices for payments, analytics, reporting, and auditing.
- Horizontal scaling of high-demand services.
- Kubernetes-based container orchestration.
- Distributed caching for improved performance.
- Advanced monitoring, tracing, and observability.
- Enhanced resilience patterns, including circuit breakers and retry mechanisms.

The current architectural decisions provide a stable foundation while allowing the platform to evolve without significant redesign.

12. Future Architectural Enhancements

The current KevaBook architecture has been designed to satisfy the functional and non-functional requirements identified for the initial release. However, the modular nature of the microservices architecture and the adoption of Domain-Driven Design (DDD) provide a solid foundation for future expansion. This section outlines potential architectural enhancements that may be introduced in subsequent versions of the platform.

12.1 Additional Business Services

As the platform evolves, additional microservices may be introduced to support new business capabilities while maintaining loose coupling and independent deployment.

Potential future services include:

Payment Service

A dedicated service responsible for processing online payments, managing refunds, and integrating with third-party payment gateways such as Stripe or PayPal.

Analytics Service

Provides business insights, including booking trends, service popularity, staff utilization, and customer activity.

Reporting Service

Generates operational and business reports for Business Owners, including revenue summaries, booking statistics, and staff performance reports.

Audit Service

Maintains an immutable audit trail of important business operations, supporting accountability, compliance, and troubleshooting.

Calendar Integration Service

Synchronizes bookings with external calendar providers such as Google Calendar and Microsoft Outlook.

12.2 Infrastructure Enhancements

Future infrastructure improvements may be implemented to improve scalability, availability, and operational efficiency.

Potential enhancements include:

- Kubernetes orchestration for container management.
- Automatic horizontal scaling of microservices.
- Load balancing across multiple service instances.
- Cloud-native deployment using managed infrastructure.
- Distributed configuration management for large-scale deployments.

12.3 Performance Enhancements

As platform usage increases, performance optimization techniques may be introduced.

Potential improvements include:

- Distributed caching using Redis.
- Database read replicas for high-volume query workloads.
- Query optimization and database indexing.
- Asynchronous processing of long-running business operations.

12.4 Security Enhancements

The current security architecture provides centralized authentication and authorization through Keycloak. Future releases may strengthen platform security through additional capabilities.

Potential enhancements include:

- Multi-Factor Authentication (MFA).
- Single Sign-On (SSO) integration with external identity providers.
- Mutual TLS (mTLS) for service-to-service communication.
- Advanced API rate limiting and abuse protection.
- Security monitoring and automated threat detection.

12.5 Observability and Monitoring

To improve operational visibility, future versions of the platform may introduce comprehensive observability capabilities.

Potential enhancements include:

- Centralized log aggregation.
- Distributed tracing across microservices.
- Application performance monitoring.
- Real-time system health dashboards.
- Automated alerting and incident management.

These capabilities will simplify troubleshooting and improve operational reliability.

12.6 Resilience Improvements

As the platform grows, additional resilience mechanisms may be introduced to improve fault tolerance.

Potential improvements include:

- Circuit breaker patterns.
- Automatic retry policies.
- Dead-letter queues for failed messages.
- Graceful degradation of non-critical services.
- Bulkhead isolation between service components.

These mechanisms will help maintain service availability during partial system failures.

12.7 Business Feature Enhancements

Future platform releases may introduce additional business functionality to better support service-based businesses.

Potential enhancements include:

- Automatic Staff assignment based on availability and workload.
- Waiting list management for fully booked schedules.
- Recurring bookings.
- Customer accounts and booking history.
- Online payment during booking.
- Promotional discounts and voucher codes.
- Customer reviews and ratings.
- Staff leave and holiday management.
- Booking reminders with configurable schedules.
- Multi-location booking support.
- Support for multiple languages and currencies.

12.8 Architectural Vision

The long-term architectural vision for KevaBook is to evolve into a scalable, secure, cloud-native Software-as-a-Service (SaaS) platform capable of supporting thousands of independent Businesses while maintaining high availability, strong tenant isolation, and independent service evolution.

The adoption of microservices, Domain-Driven Design, event-driven communication, and the Database-per-Service pattern provides a flexible architectural foundation that enables new business capabilities to be introduced with minimal impact on existing services.

Future enhancements will continue to follow the established architectural principles of service autonomy, loose coupling, scalability, maintainability, and security by design.

13. Conclusion

The KevaBook Software Architecture has been designed to provide a scalable, secure, maintainable, and extensible foundation for a modern cloud-based booking platform. The architecture adopts the microservices architectural style, enabling each business capability to be developed, deployed, and maintained independently while supporting future growth and continuous evolution.

The platform applies Domain-Driven Design (DDD) principles to align software boundaries with the core business domains of the system. Each bounded context is implemented as an autonomous microservice with clearly defined responsibilities and ownership of its business logic and persistent data. This approach promotes loose coupling, high cohesion, and clear separation of concerns across the platform.

Communication between services is achieved through a hybrid approach that combines synchronous HTTP REST interfaces with asynchronous event-driven messaging using RabbitMQ. This communication model enables responsive client interactions while supporting reliable background processing and scalable integration between services.

The Database-per-Service architectural pattern ensures that each microservice maintains exclusive ownership of its data, eliminating direct database dependencies and enabling independent schema evolution. PostgreSQL provides reliable transactional data management.

Security is implemented through a centralized authentication and authorization model using Keycloak and JSON Web Tokens (JWT). The API Gateway serves as the primary security enforcement point, validating client requests before routing them to backend services. This approach provides consistent security policies while allowing individual microservices to focus on implementing business functionality.

The deployment architecture leverages Docker-based containerization to provide consistent execution across development, testing, staging, and production environments. Supporting infrastructure components

including the API Gateway, Service Discovery, Configuration Service, RabbitMQ, and dedicated PostgreSQL databases collectively provide a robust runtime environment for the platform.

The architecture has also been designed with future expansion in mind. Its modular structure enables the introduction of new business capabilities, such as payment processing, analytics, reporting, auditing, calendar integration, and advanced scheduling features, without requiring significant changes to existing services. This flexibility ensures that the platform can evolve alongside changing business requirements while preserving architectural integrity.

Overall, the KevaBook Software Architecture provides a comprehensive and production-oriented foundation for developing a reliable multi-tenant Software-as-a-Service (SaaS) booking platform. By combining established architectural patterns, modern development technologies, and sound software engineering principles, the architecture supports the delivery of a secure, high-quality, and scalable solution capable of serving diverse service-based businesses.